**21CSE525T – AGENT TECHNOLOGY**

**ASSIGNMENT 4.2**

# Knowledge Representation and Adaptive API Integration

*A validated ontology and six live public feeds that change CipherMesh plans as the world changes*


*Submitted by*

KUMAR WAIBHAV AKSHAT [RA2212704010027]

M.Tech (Integrated) Computer Science & Engineering
with specialization in Data Science

*Under the Guidance of*

Dr. Prabhu Kavin B

Assistant Professor
Department of Data Science and Business Systems

DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR - 603 203


NOVEMBER 2026

# TABLE OF CONTENTS

**1 INTRODUCTION ..... 1**
1.1 Objective ..... 1
1.2 The problem in CipherMesh terms ..... 1
1.3 How this unit builds on Units 1-3 ..... 1
**2 ASSIGNMENT REQUIREMENTS AND TRACEABILITY ..... 2**
**3 DESIGN ..... 2**
3.1 Architecture ..... 2
3.2 Ontology ..... 3
3.3 Data model ..... 5
3.4 Algorithms ..... 5
**4 IMPLEMENTATION ..... 8**
4.1 Modules ..... 8
4.2 Key code ..... 9
**5 VERIFICATION RUN ..... 11**
5.1 Set-up ..... 11
5.2 Phase A: knowledge base and default plans ..... 11
5.3 Phase B: live endpoints ..... 11
5.4 Property changes and timing ..... 13
5.5 Phase C: the same prompts after the live update ..... 14
5.6 Phase D: deterministic replay (not live) ..... 15
**6 TESTING ..... 17**
**7 SUBMISSION ANALYSIS ..... 18**
7.1 Strengths ..... 18
7.2 Limitations ..... 18
7.3 How this feeds the Capstone (5.2) ..... 19
**8 CONCLUSION ..... 19**
**REFERENCES ..... 20**
**App. A HOW TO RUN ..... 21**
**App. B FILE LISTING ..... 22**

# LIST OF FIGURES

Figure 3.1 Live knowledge pipeline from public endpoint to agent plan....3
Figure 3.2 CipherMesh ontology: main classes and typed relations....4
Figure 3.3 KEV backtest: 30-day ratio, watched-vendor hits and the calibrated posture over one year. 8
Figure 5.1 Fetch latency and payload size of the six live endpoints (real run)....12
Figure 5.2 Fetch, parse, property-change and re-plan latency per live feed (log scale)....13
Figure 5.3 Route utility and chosen provider per prompt, Phase A versus Phase C....14
Figure 5.4 Provider placement and status sub-graph exported by the agent after the replay....16

# LIST OF TABLES

Table 1.1 Earlier CipherMesh units reused by Assignment 4.2....1
Table 2.1 Requirement traceability....2
Table 3.1 Main individuals in ontology.json....3
Table 3.2 Run-time data model....5
Table 3.3 KEV backtest and calibrated thresholds (logs/kev_backtest.json)....7
Table 4.1 Modules of ciphermesh/knowledge/....8
Table 4.2 Agent properties and their sources....10
Table 5.1 Live endpoints contacted in Phase B....12
Table 5.2 Knowledge extracted from the live feeds....12
Table 5.3 Adaptation latency per live feed (ms)....14
Table 5.4 Plans before (Phase A) and after (Phase C) the live update....14
Table 5.5 Attribution: each feed removed in turn (same parsed data, re-planned)....15
Table 5.6 Replay: plans at each step....15
Table 6.1 New tests in tests/knowledge/....17
Table 7.1 Requirement status....18
Table B.1 Deliverable files....22

# 1 INTRODUCTION

## 1.1 Objective

This assignment gives CipherMesh, the privacy control plane that sits between enterprise applications and AI providers, a **formal knowledge base** and connects that knowledge base to **live public data**. The brief asks for three things: a structured format that stores the agent's domain knowledge, an integration with real-world REST endpoints that feed the agent real knowledge, and a parsing module that changes the agent's properties and plans as soon as new live data arrives.

The knowledge base is an ontology written as a JSON-LD 1.1 style document [1] with classes, individuals and typed relations in the spirit of OWL 2 [2] and RDF Schema [3]. Following Gruber's definition of an ontology as an explicit specification of a shared conceptualisation [4], every fact the router and the policy engine previously kept as Python constants (where each provider runs, which data categories are regulated, what each rule decides) is now stated once, in a file that a program can validate and reason over [5].

## 1.2 The problem in CipherMesh terms

Up to Unit 3.2 CipherMesh planned with frozen assumptions. The router always believed that every provider was healthy, that prices in US dollars matched an Indian budget, and that the security climate never changed. The rule "Aadhaar, PAN or an Indian phone number may only go to an Indian or local provider" was a hard-coded Python set. None of this is true in production. Vertex AI has incidents, the rupee moves, and new exploited vulnerabilities appear every week. A privacy gateway that ignores these facts either sends data to a degraded endpoint or keeps a loose policy while attackers are busy.

This unit answers the question: *how can the agent learn about the outside world, store that knowledge with its source and age, and change its plan at once?* The answer has four parts: an ontology, six live connectors, an adapter that turns parsed facts into properties, and an adaptive agent that re-plans whenever a property changes.

## 1.3 How this unit builds on Units 1-3

Table 1.1 shows what is reused. Nothing in the engine was rewritten; the new subpackage `ciphermesh/knowledge/` calls the existing modules, and all 112 engine tests still pass.

**Table 1.1:** Earlier CipherMesh units reused by Assignment 4.2

| Unit | Engine part | How 4.2 uses it |
|---|---|---|
| 1.1 / 1.2 | PEAS model, detector (25 rules, D1-D12), policy R1-R5 | Every detector category and rule is an ontology individual; `plan_for` calls the real `inspect_text` |
| 2.1 | Threads, locks, daemon runtime | `AgentProperties` and `KnowledgeGraph` are thread-safe (RLock) and tested with concurrent writers |
| 2.2 | Typed wire models, `AgentState`, `CipherMeshAgent` | Derived policy settings are written into a real `AgentState`; `sync_engine_agent` pushes them into a running agent |
| 3.1 | Contract-Net procurement | Not used in this unit's demo (list prices). Capstone hook: negotiated prices enter the same `route` call through `contract_prices`, under the live FX budget |
| 3.2 | Constrained utility router U = 0.30Q + 0.25C + 0.20L + 0.15A + 0.10P | Called unchanged; live knowledge changes its inputs: profiles, budget, residency, exclusions |

# 2 ASSIGNMENT REQUIREMENTS AND TRACEABILITY

Table 2.1 quotes each line of the brief word for word and maps it to the code that satisfies it and to the evidence file a reviewer can open. The last row is an extra design requirement set for this project; it is not part of the brief.

**Table 2.1:** Requirement traceability

| Brief line (verbatim) | Where satisfied | Evidence |
|---|---|---|
| Objective: "Enhance your agent's knowledge base through semantic knowledge representation or integration with external data APIs." | `ciphermesh/knowledge/` (ontology + live feeds) | Sections 3-5; `logs/live_feed_run.log` |
| Direction: "Implement a structured configuration format (JSON, Dictionary objects, or an ontological graph) to formally store your agent's domain knowledge." | `ontology.json` (JSON-LD), `ontology.schema.json`, `ontology.py`, `graph.py` | `test_ontology.py`, `test_graph.py`; Phase A of the run log |
| Direction: "Integrate an external real-world REST API to feed the agent real knowledge maps (e.g., real live OpenWeather API maps for Travel planners, or real Geolocation maps for WiFi searches)." | `knowledge/feeds/`: six connectors on public endpoints | `logs/live_endpoints_report.json`, `data/snapshots/*.meta.json` |
| Direction: "Enable the agent to dynamically adapt its internal plans based on the live unstructured data ingested." | `feeds/ai_incidents.py`, `normalize.py`, `adapter.py`, `posture.py`, `adaptive_agent.py` | Phase C diff and leave-one-feed-out attribution; `test_adaptive_agent.py` |
| Deliverables: "Code integration module demonstrating active connection to live endpoints and a parsing module changing agent properties instantly based on live feeds." | `feeds/base.py` + connectors; parsers, `KnowledgeAdapter`, `AgentProperties` | Phase B log; `logs/adaptation_events.jsonl` (property changes, latencies) |
| Additional design requirement (not in brief): residency rule derivable from the graph | `KnowledgeGraph.derive_residency` | Equivalence tests (25 categories, 300 pairs); `EQUAL=True` in the log |

# 3 DESIGN

## 3.1 Architecture

Figure 3.1 shows the live data path. Each public endpoint has one connector. Connectors share a hardened HTTP client, a time-to-live cache and a snapshot store. Their raw bytes go through a safe parser and a normaliser whose vocabulary comes from the ontology itself. The adapter turns every parsed **fact** into a graph triple and every **signal** into an agent property, then evaluates two derived rules whose thresholds are also stored in the ontology. The adaptive agent subscribes to the properties and rebuilds its plan context inside the same call that changed the property.

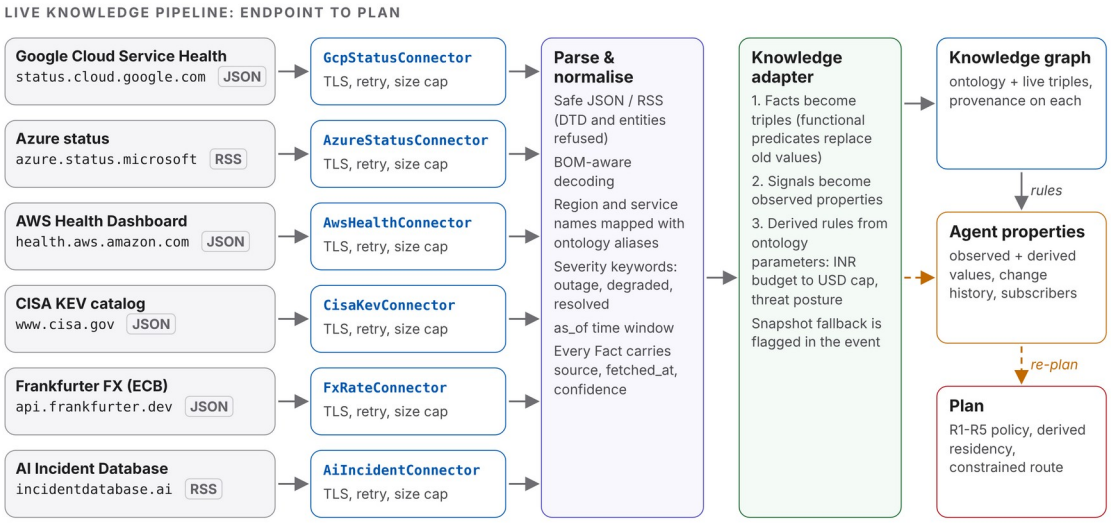



**Figure 3.1:** Live knowledge pipeline from public endpoint to agent plan

The design keeps three kinds of state apart. The **knowledge graph** holds what the agent knows, with provenance. The **properties** hold the few control values that plans depend on. The **plan context** is derived from both and is never edited by hand. This separation makes every plan change explainable: a changed plan always has a property change behind it, and every property change names its source feed and confidence.

## 3.2 Ontology

Table 3.1 lists the main individuals of the ontology, and Figure 3.2 shows its 16 most important classes with the typed relations between them. Solid arrows are asserted in `ontology.json`; dashed orange arrows are relations whose value is replaced at run time from live feeds. Sub-classes are listed inside the boxes (for example `ResidencyPolicy` is a sub-class of `OrganisationalPolicy`, which is a sub-class of `LegalInstrument`).

The file declares 29 classes, 24 object properties and 44 datatype properties, and 198 subjects in total. After loading, the graph holds 1,170 triples: 1027 asserted and 143 inferred by RDFS rules. Every provider individual matches the engine profile in `providers.py` field by field, and a test checks this.

The ontology is validated in two layers. The JSON Schema (draft 2020-12 [6]) checks structure: required header fields, a semantic version, CURIE syntax for every identifier, and the allowed value shapes. Semantic validation then checks what a schema cannot express: every referenced IRI is defined, every object-property value is an instance of the declared `rdfs:range`, functional properties have one value per subject, and there are no sub-class cycles.

**Table 3.1:** Main individuals in ontology.json

| Class | Count | Individuals (examples) |
|---|---|---|
| Provider | 4 | aws-bedrock (ap-south-1, IN), azure-ai (centralindia, IN), gcp-vertex (us-central1, US), local (on-host, LOCAL) |
| CloudRegion / Jurisdiction | 4 / 4 | ap-south-1, centralindia, us-central1, on-host / IN, US, EU, LOCAL |
| DataCategory / DetectorFamily | 25 / 12 | aadhaar memberOf D10, pan memberOf D11, phone_in memberOf D6, llm_api_key memberOf D4 ... |
| LegalInstrument | 4 | DPDP Act 2023 [7], Aadhaar Act 2016 [8], GDPR [9], CipherMesh India residency policy |

| Class | Count | Individuals (examples) |
|---|---|---|
| PolicyRule / Decision | 6 / 4 | LOCKDOWN, R5, R1, R3, R2, R4 (priority + precedes) / ALLOW, REDACT, REVIEW, BLOCK |
| Threat / ThreatPosture | 6 / 3 | credential-leak, pii-exposure, cross-border-transfer ... / LOW, ELEVATED, HIGH with thresholds |
| InformationSource / Vendor | 6 / 14 | gcp-status, azure-status, aws-health, cisa-kev, frankfurter, aiid / Amazon, Microsoft, Google, Linux ... |

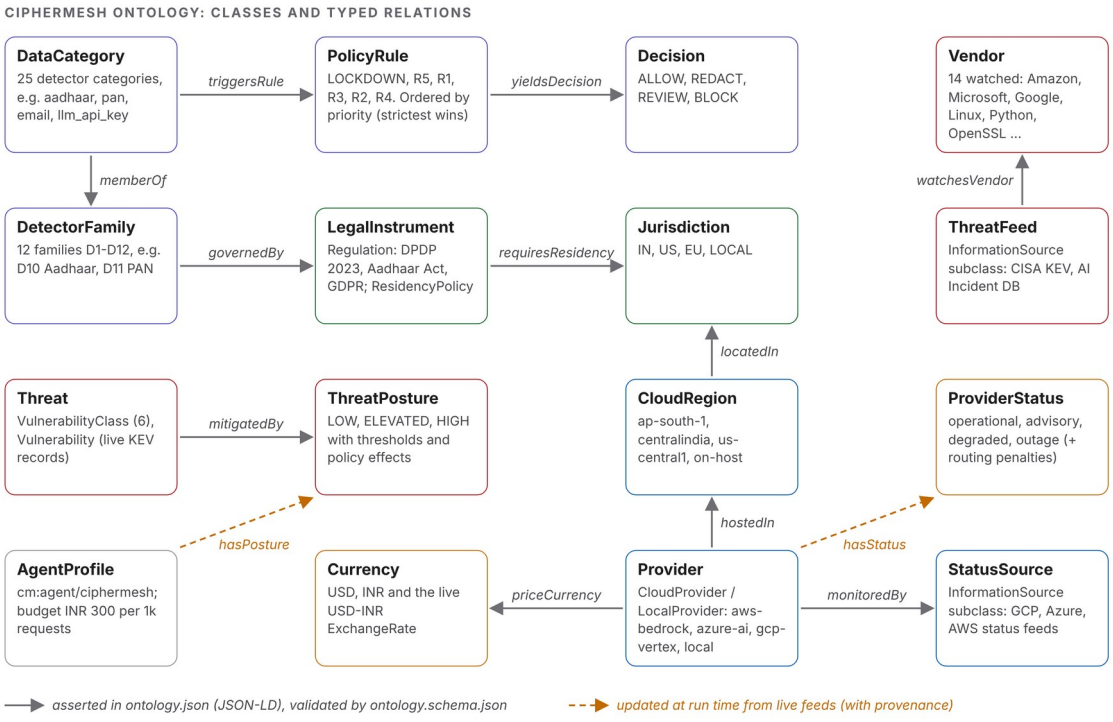



**Figure 3.2:** CipherMesh ontology: main classes and typed relations

Listing 1 is the real provider node for AWS Bedrock. The `@context` declares `hostedIn` with `"@type": "@id"`, so the string value is read as a link to the region node; `costPer1kUsd` is declared as `xsd:decimal`, so 2.8 becomes a typed literal. This is JSON-LD type coercion [1], and it lets the file stay plain JSON [10] while still meaning a graph.

```
{
 "@id": "cm:provider/aws-bedrock",
 "@type": "cm:CloudProvider",
 "label": "AWS Bedrock",
 "providerId": "aws-bedrock",
 "kind": "cloud",
 "hostedIn": "cm:region/ap-south-1",
 "operatedBy": "cm:vendor/amazon",
 "monitoredBy": "cm:feed/aws-health",
 "priceCurrency": "cm:cur/USD",
 "costPer1kUsd": 2.8,
 "p95LatencyMs": 650,
 "qualityScore": 4.6,
 "availabilityPct": 99.9,
 "hasStatus": "cm:status/operational",
 "serviceAlias": [
  "Amazon Bedrock",
  "Bedrock",
  "bedrock"
 ]
},
```

*Listing 1: provider individual in ontology.json*

## 3.3 Data model

Table 3.2 lists the run-time data structures. Each one carries enough provenance to answer "where did this come from and when?".

**Table 3.2:** Run-time data model

| Structure | Fields | Purpose |
|---|---|---|
| `Provenance` | source, fetched_at, confidence, method (asserted / live / inferred / derived), evidence | Attached to every triple in the graph |
| `Fact` | subject, predicate, obj, obj_is_iri, source, fetched_at, confidence, evidence | One statement extracted by a parser |
| `ParseReport` | items_seen, items_relevant, facts, signals, warnings, parse_ms | Parser output for one feed |
| `FetchResult` | url, final_url, mode (live / cache / snapshot / replay-file), status, bytes, sha256, original_sha256, latency_ms, fetched_at, age_s, stale | Network evidence for one request |
| `PropertyChange` | seq, key, old, new, source, confidence, at, mono_ns | One entry in the property history |
| `AdaptationEvent` | fetch evidence, facts, triples added / replaced, property changes, plan versions, timings_ms | One line in `adaptation_events.jsonl` |
| `PlanContext` / `Plan` | effective profiles, constraints, policy state, posture / decision, route, utility, reasons | What the agent will do with a prompt |

## 3.4 Algorithms

### *A1. Residency derivation from the graph*

1. Detect categories in the prompt with the engine detector (for example `aadhaar`, `email`).
2. For each category, collect the category node and the family it is `memberOf`.

3. For each of these holders, follow `governedBy` to legal instruments. Keep only instruments that are (by inference) a `ResidencyPolicy` and that have `requiresResidency` values.
4. Map the required jurisdictions to codes and intersect them across all applicable policies (strictest wins, so adding data can only narrow the set).
5. Return the allowed jurisdictions, the list of allowed providers (provider -> hostedIn -> locatedIn) and a human-readable inference chain.

Only `aadhaar` (via family D10), `pan` (via family D11) and `phone_in` (directly) reach the residency policy, so the derived set is exactly the engine's `INDIA_PERSONAL_DATA`. A test checks this for all 25 single categories and all 300 pairs. Regulations such as the DPDP Act and GDPR are also linked to categories, but they carry no residency requirement, which matches the law: Section 16 of the DPDP Act allows transfers except to countries the government restricts [7]. The India-only rule is therefore modelled honestly as an organisational policy that is `basedOn` the statutes.

### *A2. Fetch, parse, then snapshot (with fallback)*

1. Refuse any URL that is not HTTPS (plain HTTP is accepted only for loopback test servers). A redirect handler applies the same check to every hop and refuses redirects to another host; the final URL is recorded.
2. Return the cached response if it is younger than the feed TTL from the ontology (`cm:ttlSeconds`).
3. Send a GET with a descriptive User-Agent, TLS 1.2 or newer and certificate verification, a per-attempt timeout and an overall deadline (60 s); read the body in 64 KiB chunks up to a size cap (8 MiB).
4. Retry network errors and HTTP 429 / 5xx with exponential back-off; do not retry other 4xx codes [11].
5. **Parse first.** Only a payload that parses is stored. Feeds with third-party prose keep a text-free *derivative* (structured fields plus the tokens the parser read); the meta file records the original URL, status, bytes, SHA-256 and fetch time. The exchange rate (a number and a date) is stored raw.
6. If the network fails or a 200 response cannot be parsed (for example an HTML error page), load the previous snapshot, verify its hash and mark the result as a fallback. A snapshot older than `cm:maxSnapshotAgeSeconds` (24 h for status feeds, 7 days otherwise) is marked stale and its confidence is multiplied by 0.4, which puts it below the 0.5 acceptance gate of the properties.

### *A3. Parsing messy status text*

1. Decode safely: honour UTF-8 and UTF-16 byte-order marks; refuse XML that contains a DTD or entity declaration (billion-laughs and XXE defence [12]).
2. Keep only records inside the time window: active at `as_of`, or resolved within the feed's look-back days.
3. Clean HTML, decode entities, apply Unicode NFKC and collapse spaces.
4. Find regions: structured location fields first, then vendor region-code patterns, then display names and aliases taken from the ontology ("Central India", "Asia Pacific (Mumbai)"); zones such as `us-central1-b` collapse to their region.
5. Find services through the provider's `cm:serviceAlias` values, and classify severity with keyword classes (outage, degraded, resolved).

6. A record affects a provider only if both the service and the provider's region match (confidence 0.85-0.9); a "global" or "multiple regions" scope is accepted with confidence 0.6.

### *A4. Derived rules: budget and threat posture*

1. Budget: `max_cost_per_1k_usd = budgetPer1k (INR, from the agent profile) / live USD-INR rate`.
2. Posture evidence is grouped into three independent signals: **KEV volume** (30-day additions divided by the median of the 12 previous 30-day windows), **KEV stack** (watched-vendor, AI/ML-product or ransomware-linked additions) and **AI incidents** (privacy-related reports per covered day; the feed lists only its newest 100 items, so the count is divided by the days those items actually cover).
3. A group is met when one of its values reaches the posture's threshold. ELEVATED needs one group, HIGH needs two (`cm:minSignals`). Postures are visited from the highest level down; LOW is the default.
4. Hysteresis: while the agent is already at a level, thresholds for that level and below are multiplied by `cm:hysteresisFactor` (0.9), so the posture does not flap.
5. The chosen posture's effects come from the ontology: R5 byte limit, extra redaction patterns, residency limit for all traffic, and a lockdown recommendation. Lockdown is never applied automatically; a human decides.

The KEV thresholds were calibrated, not guessed. The parser was run on the pinned catalog every 3 days from 2025-11-01 to 2026-10-30 (122 dates), and each threshold is the smallest value reached on at most 10 % (ELEVATED) or 3 % (HIGH) of those dates. Table 3.3 shows the result and Figure 3.3 the posture the rule would have produced. The first, uncalibrated thresholds put the agent in HIGH on 46 % of days, which made the signal useless; the calibrated rule is HIGH on 0.8 % and ELEVATED on 23 %. The AI-incident threshold (0.5 privacy reports per day for ELEVATED, 1.0 for HIGH) cannot be backtested because the feed has no history; it is a stated judgement value and can supply only one of the two signals HIGH needs.

**Table 3.3:** KEV backtest and calibrated thresholds (logs/kev_backtest.json)

| Signal | p50 | p90 | p97 | Max | ELEVATED >= | HIGH >= | Days >= E / H |
|---|---|---|---|---|---|---|---|
| KEV volume ratio | 1.25 | 1.683 | 1.88 | 2.043 | 1.69 | 1.89 | 9.8 / 2.5 % |
| Watched-vendor hits | 5 | 10 | 11 | 12 | 11 | 12 | 5.7 / 0.8 % |
| AI/ML-product hits | 0 | 2 | 2 | 3 | 3 | 4 | 1.6 / 0.0 % |
| Ransomware-linked hits | 0 | 1 | 3 | 3 | 2 | 4 | 8.2 / 0.0 % |
| Posture share (uncalibrated) | | | | | 35 % | 46 % | LOW 19 % |
| Posture share (calibrated) | | | | | 23 % | 0.8 % | LOW 76 % |

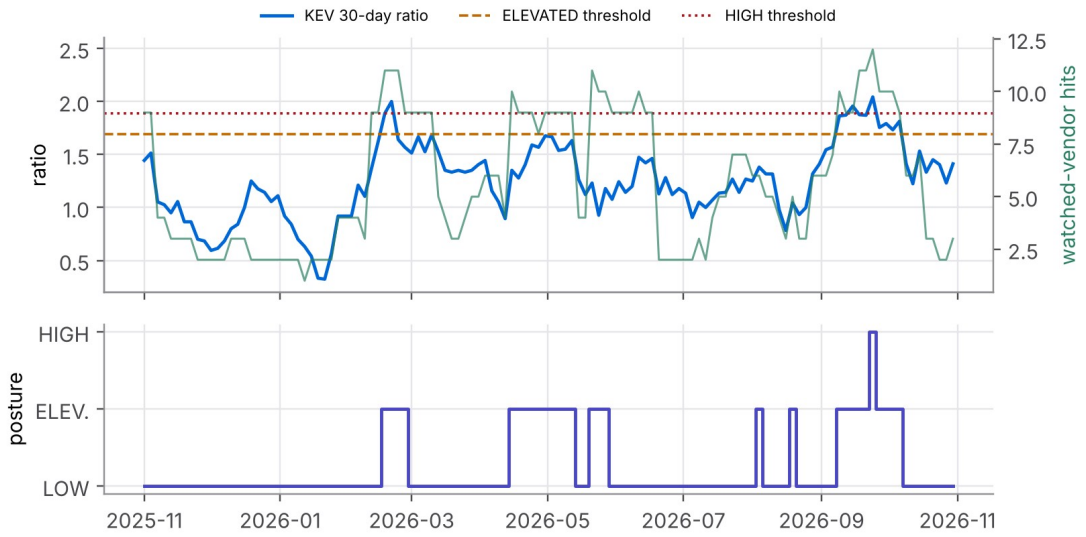



**Figure 3.3:** KEV backtest: 30-day ratio, watched-vendor hits and the calibrated posture over one year

### *A5. Reactive re-planning*

`AgentProperties.set` stores the value, appends a `PropertyChange` and calls subscribers synchronously. The adaptive agent's subscriber rebuilds the plan context: provider profiles adjusted by status (`cm:availabilityPenalty`, `cm:latencyFactor`, `cm:excludesProvider` from the status individual), the policy state (byte limit, custom patterns re-checked by the engine's own regex guardrail), and the routing constraints (budget, residency). The latency from the property change to the new plan is measured with a monotonic clock.

# 4 IMPLEMENTATION

## 4.1 Modules

Table 4.1 lists the new subpackage. It adds 2,963 lines of source code and 1,243 lines of tests; the engine modules from Units 1-3 are untouched.

**Table 4.1:** Modules of ciphermesh/knowledge/

| File | Purpose | Key classes / functions |
|---|---|---|
| `ontology.json`, `ontology.schema.json` | Formal knowledge base and its JSON Schema | 198 nodes; draft 2020-12 schema |
| `ontology.py` | Load, validate, compile to triples | `load_graph`, `schema_errors`, `semantic_errors`, `summary` |
| `graph.py` | Triple store with provenance, inference and export | `KnowledgeGraph`, `Literal`, `Provenance`, `derive_residency`, `to_svg` |
| `properties.py` | Property sink protocol and observable store | `PropertySink`, `AgentProperties`, `PropertyChange` |
| `adapter.py` | Facts to triples, signals to properties, derived rules | `KnowledgeAdapter.ingest`, `derive`, `AdaptationEvent` |
| `posture.py`, `calibration.py` | Posture rule (signal groups, hysteresis, attribution); KEV backtest | `decide`, `attribution`, `kev_series`, `calibrate` |

| File | Purpose | Key classes / functions |
|---|---|---|
| `adaptive_agent.py` | Agent wrapper that re-plans on change | `AdaptiveCipherMeshAgent.plan_for`, `update_from`, `replay_file` |
| `feeds/base.py` | HTTP client, cache, snapshots, connector base | `HttpFetcher`, `TTLCache`, `SnapshotStore`, `FeedConnector` |
| `feeds/safe_xml.py`, `normalize.py` | Hostile-input-safe RSS; text normalisation | `parse_rss`, `UnsafeXMLError`, `Normaliser`, `classify_impact` |
| `feeds/gcp_status.py`, `azure_status.py`, `aws_health.py` | Provider status connectors | `GcpStatusConnector`, `AzureStatusConnector`, `AwsHealthConnector` |
| `feeds/cisa_kev.py`, `fx_rates.py`, `ai_incidents.py` | Threat and market connectors | `CisaKevConnector`, `FxRateConnector`, `AiIncidentConnector` |

## 4.2 Key code

Listing 2 is the core of the residency derivation in `graph.py`. It walks the graph instead of reading a Python constant, so adding a new regulated category is a one-line change in the ontology.

```
    for name in cats:
        node = self.category_id(name)
        holders = [node] + self.objects(node, "cm:memberOf")
        for holder in holders:
            for inst in self.objects(holder, "cm:governedBy"):
                jurs = self.objects(inst, "cm:requiresResidency")
                if not jurs or not self.is_instance(inst, "cm:ResidencyPolicy"):
                    continue
                codes = {str(self.value(j, "cm:jurisdictionCode")) for j in jurs}
                policies.add(inst)
                if name not in restricted:
                    restricted.append(name)
                via = "" if holder == node else f" (via family {holder})"
                chain.append(f"{node}{via} --governedBy--> {inst} --requiresResidency--> "
                             f"{sorted(codes)}")
                allowed = codes if allowed is None else allowed & codes
    provs = []
```

*Listing 2: residency derivation (graph.py)*

Listing 3 shows how the threat posture is chosen. Thresholds, the number of independent signals and the hysteresis factor are read from the ontology, so the policy can be tuned without changing code.

```
for spec in specs:
    if spec.level == 0:
        continue
    factor = hysteresis if spec.level <= current_level else 1.0
    met: dict[str, list[str]] = {}
    for group, pairs in SIGNAL_GROUPS.items():
        hits = []
        for attr, key in pairs:
            t, v = spec.thresholds.get(attr), evidence.get(key)
            if t is None or v is None:
                continue
            if float(v) >= t * factor:
                hits.append(f"{key.split('.', 1)[1]}={v} >= {round(t * factor, 3)}")
        if hits:
            met[group] = hits
    if len(met) >= spec.min_signals:
        return PostureDecision(spec, met)
return PostureDecision(specs[-1], {})
```

*Listing 3: posture rule (posture.py)*

Listing 4 shows the two-pass XML guard. The raw bytes are rejected if they contain a DTD, and the expat handlers raise as a second layer before the element tree is built.

```
def parse_xml(body: bytes) -> ET.Element:
    if len(body) > MAX_XML_BYTES:
        raise UnsafeXMLError(f"XML document larger than {MAX_XML_BYTES} bytes")
    if _FORBIDDEN.search(body):
        raise UnsafeXMLError("XML document contains a DTD or entity declaration; refused")
    guard = expat.ParserCreate()            # pass 1: raw expat with refusing handlers
    guard.StartDoctypeDeclHandler = _refuse
    guard.EntityDeclHandler = _refuse
    guard.UnparsedEntityDeclHandler = _refuse
    guard.ExternalEntityRefHandler = _refuse
    try:
        guard.Parse(body, True)
        return ET.fromstring(body)          # pass 2: build the tree (no DTD present)
    except (expat.ExpatError, ET.ParseError) as exc:
        raise ValueError(f"malformed XML: {exc}") from exc
```

*Listing 4: refusing DTDs and entities (safe_xml.py)*

Table 4.2 lists the agent properties written by the adapter. Observed properties come straight from a feed; derived properties name the rule or ontology node that produced them.

**Table 4.2:** Agent properties and their sources

| Property key | Kind | Source | Value after live run |
|---|---|---|---|
| `provider.<id>.status` | observed | gcp / azure / aws status feeds | operational (x3) |
| `fx.usd_inr` | observed | Frankfurter (ECB rates) | 94.91 |
| `threat.kev.ratio`, `.relevant_hits` | observed | CISA KEV | 1.404, 3 |
| `threat.aiid.privacy_incidents` / `_per_day` | observed | AI Incident Database | 11 / 0.5819 |
| `router.max_cost_per_1k_usd` | derived | budget rule | 3.1609 |
| `threat.posture` | derived | posture rule | ELEVATED |
| `policy.max_prompt_bytes` | derived | cm:posture/ELEVATED | 32768 |
| `policy.custom_redaction_patterns` | derived | cm:posture/ELEVATED | 2 (ARN, k8s DNS) |
| `policy.lockdown_recommended` | derived | cm:posture/ELEVATED | false |

**PROPERTYSINK AND THE CAPSTONE**

The adapter depends only on `set(key, value, source, confidence=1.0) -> bool` and `get(key, default=None)`. This is the same shape as the BDI belief base of Assignment 4.1 [13], so in the Capstone (5.2) live feeds can write beliefs directly and trigger intention reconsideration. A test checks that a belief-base-like class satisfies the protocol.

# 5 VERIFICATION RUN

## 5.1 Set-up

The demonstration was run once with `python demo/live_adaptation_demo.py` on 2026-11-02 (UTC). All network traffic used HTTPS (TLS 1.2+, certificate verification on), urllib, timeout 20 s, 2 retries, 8 MiB cap. The run prints a timestamped log (`logs/live_feed_run.log`), writes one JSON line per adaptation (`logs/adaptation_events.jsonl`) and a per-endpoint report (`logs/live_endpoints_report.json`), and keeps snapshots in `data/snapshots/`. Five prompts are planned in each phase: a clean request, a request with an email address, a KYC request with an Aadhaar number, a question containing an AWS ARN, and a 40 KB service log.

## 5.2 Phase A: knowledge base and default plans

Listing 5 is the start of the run. The ontology passes the schema, and the residency rule derived from the graph equals the engine's hard-coded rule.

```
21:42:14.652 ontology.json validated against ontology.schema.json: 0 schema errors;
      loaded + inferred in 379.8 ms
21:42:14.653   knowledge  version            4.2.0
21:42:14.653   knowledge  triples            1170
21:42:14.653   knowledge  by_asserted        1027
21:42:14.653   knowledge  by_inferred        143
21:42:14.653   knowledge  providers          4
21:42:14.653   knowledge  data_categories    25
21:42:14.653   knowledge  live_sources       6
21:42:14.654   residency rule derived from graph : ['aadhaar', 'pan', 'phone_in'] ->
      ['IN', 'LOCAL']
21:42:14.654   engine hard-coded INDIA_PERSONAL_DATA: ['aadhaar', 'pan', 'phone_in']
      -> ['IN', 'LOCAL']  EQUAL=True
21:42:14.654   inference chain: cm:cat/aadhaar (via family cm:family/D10)
      --governedBy--> cm:policy/india-residency --requiresResidency--> ['IN', 'LOCAL']
21:42:14.654   inference chain: cm:cat/phone_in --governedBy-->
      cm:policy/india-residency --requiresResidency--> ['IN', 'LOCAL']
```

*Listing 5: Phase A excerpt from logs/live_feed_run.log*

## 5.3 Phase B: live endpoints

Table 5.1 is taken from `live_endpoints_report.json`. Every endpoint answered HTTP 200 over verified TLS. The hash in the table is the SHA-256 of the payload as served. Five of the feeds contain third-party prose (incident descriptions, vulnerability write-ups, news headlines), so their raw payloads are not redistributed: each snapshot keeps the original hash in its meta file and stores only a text-free derivative that parses to exactly the same signals (a test checks this for every feed). No parser copies feed headlines or descriptions into the graph, the properties or the logs; incident labels are built from ids and codes.

**Table 5.1:** Live endpoints contacted in Phase B

| Feed | Status | Bytes | Fetch ms | Parse ms | Items / relevant | Facts | SHA-256 (prefix) |
|---|---|---|---|---|---|---|---|
| gcp_status | 200 | 251,117 | 426.8 | 5.63 | 8 / 0 | 1 | 5e979a2555c2 |
| azure_status | 200 | 577 | 275.1 | 0.31 | 0 / 0 | 1 | 439d001adcdd |
| aws_health | 200 | 231,276 | 660.2 | 5.27 | 2 / 0 | 11 | ead64c82685d |
| cisa_kev | 200 | 1,826,755 | 589.1 | 11.37 | 1766 / 3 | 19 | 3aa9cf3c01f1 |
| fx_usd_inr | 200 | 69 | 161.9 | 0.11 | 1 / 1 | 2 | 750d255e986c |
| aiid_rss | 200 | 88,482 | 298.3 | 10.01 | 100 / 18 | 72 | 6e40f9f733b4 |

Figure 5.1 plots the same run. Payloads range from 69 bytes (the exchange rate) to 1.8 MB (the full KEV catalog); fetch latency stayed under one second for every feed. Table 5.2, placed first, shows what each parser extracted from the live data.

**Table 5.2:** Knowledge extracted from the live feeds

| Feed | What the parser found | Effect on the agent |
|---|---|---|
| gcp_status | 8 incidents; none active; 2 resolved within 30 days (europe-west2; a Cloud Storage incident in us-central1 that does not name Vertex AI) | gcp-vertex = operational (confidence 0.8) |
| azure_status | Empty channel: no active Azure incident | azure-ai = operational (0.8) |
| aws_health | 2 events, both in the Middle East (me-central-1, me-south-1); none in ap-south-1 | aws-bedrock = operational (0.8); incidents stored as graph facts |
| cisa_kev | Catalog 2026.10.30: 33 additions in 30 days vs median 23.5 (ratio 1.404); watched vendors: Linux Kernel, Google Brotli, Microsoft Windows | below the calibrated thresholds (KEV alone gives LOW) |
| fx_usd_inr | USD/INR = 94.91 (ECB reference date 2026-11-02) [14] | budget USD 3.1609 per 1k |
| aiid_rss | 100 reports covering 18.9 days; keyword classes in free text: 11 privacy, 10 intrusion (report text is not stored) | 0.5819 privacy reports per day >= 0.5: posture ELEVATED |

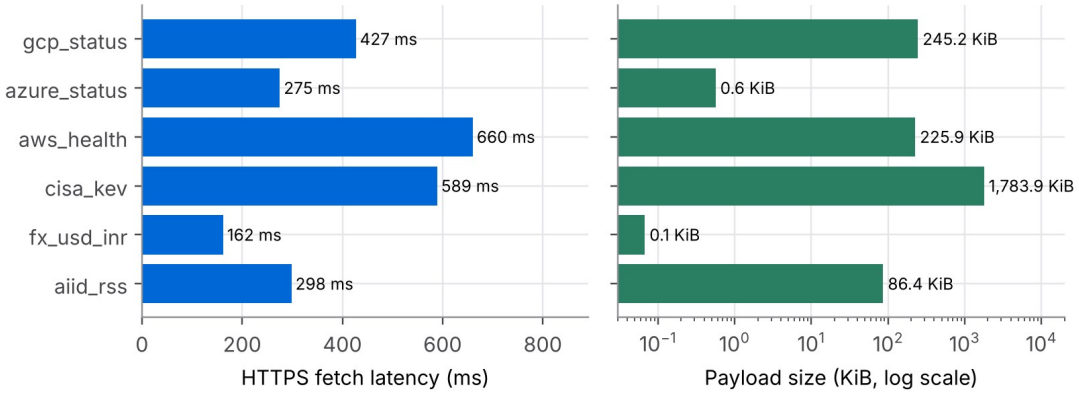



**Figure 5.1:** Fetch latency and payload size of the six live endpoints (real run)

Listing 6 is the real log of the AI Incident Database update: unstructured news text, classified by keywords, raises the posture. One fetch produces seven property changes: two observations, then the posture rule and its effects. Each change carries its source, confidence and time.

```
21:42:17.220   [aiid_rss] LIVE url=https://incidentdatabase.ai/rss.xml
21:42:17.221       http=200 bytes=88482 fetch=298.3ms parse=10.01ms
      sha256=6e40f9f733b405cb... fetched_at=2026-11-02T21:42:17.195+00:00
21:42:17.221       items=100 relevant=18 facts=72 triples_added=72 replaced=0 plan
      v21->v28
21:42:17.221       property threat.aiid.privacy_incidents: None -> 11
      (src=aiid_rss:live, conf=0.7, at=2026-11-02T21:42:17.218+00:00)
21:42:17.221       property threat.aiid.privacy_per_day: None -> 0.5819
      (src=aiid_rss:live, conf=0.7, at=2026-11-02T21:42:17.218+00:00)
21:42:17.221       property threat.posture: 'LOW' -> 'ELEVATED'
      (src=derived:posture-rule, conf=1.0, at=2026-11-02T21:42:17.219+00:00)
21:42:17.221       property threat.posture_reason: 'no signal group meets an ELEVATED
      threshold' -> 'ai_incidents: aiid.privacy_per_day=0.5819 >= 0.5'
      (src=derived:posture-rule, conf=1.0, at=2026-11-02T21:42:17.219+00:00)
21:42:17.221       property threat.posture_signals: '[0 items]' -> '[1 items]'
      (src=derived:posture-rule, conf=1.0, at=2026-11-02T21:42:17.219+00:00)
21:42:17.221       property policy.max_prompt_bytes: 65536 -> 32768
      (src=derived:cm:posture/ELEVATED, conf=1.0, at=2026-11-02T21:42:17.219+00:00)
21:42:17.221       property policy.custom_redaction_patterns: '[0 items]' -> '[2
      items]'  (src=derived:cm:posture/ELEVATED, conf=1.0,
      at=2026-11-02T21:42:17.219+00:00)
21:42:17.221       latency: fetch->parsed 309.046ms | parsed->property 13.035ms |
      property->plan 0.799ms | end-to-end 324.624ms
```

*Listing 6: AI Incident Database update in Phase B*

## 5.4 Property changes and timing

Figure 5.2 shows the timing chain for each live feed, read from `adaptation_events.jsonl`. The network fetch dominates (hundreds of milliseconds). Parsing takes from 0.1 ms (exchange rate) to about 12 ms (1.8 MB KEV catalog). The step that matters for "instant" adaptation, from a property change to a recomputed plan, stays well under one millisecond for every feed. Table 5.3 gives the exact numbers.

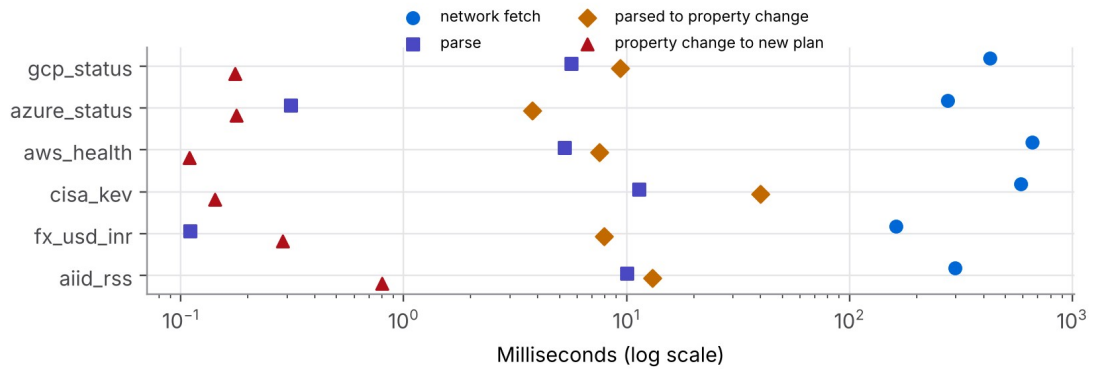



**Figure 5.2:** Fetch, parse, property-change and re-plan latency per live feed (log scale)

**Table 5.3:** Adaptation latency per live feed (ms)

| Feed | Fetch | Parse | Parsed to property | Property to plan | End to end | Plan version |
|---|---|---|---|---|---|---|
| gcp_status | 426.8 | 5.63 | 9.354 | 0.175 | 443.4 | v1 -> v3 |
| azure_status | 275.1 | 0.31 | 3.766 | 0.177 | 280.6 | v3 -> v5 |
| aws_health | 660.2 | 5.27 | 7.536 | 0.109 | 674.1 | v5 -> v7 |
| cisa_kev | 589.1 | 11.37 | 39.847 | 0.142 | 644.8 | v7 -> v18 |
| fx_usd_inr | 161.9 | 0.11 | 7.921 | 0.287 | 171.6 | v18 -> v21 |
| aiid_rss | 298.3 | 10.01 | 13.035 | 0.799 | 324.6 | v21 -> v28 |

## 5.5 Phase C: the same prompts after the live update

Table 5.4 compares the plans and Figure 5.3 shows the same data as a chart. Every one of the five prompts was planned differently, and every difference is traced to a named live fact.

**Table 5.4:** Plans before (Phase A) and after (Phase C) the live update

| Prompt | Phase A | U | Phase C | U | Why it changed |
|---|---|---|---|---|---|
| P1 clean | ALLOW -> gcp-vertex | 76.22 | ALLOW -> local | 69.74 | budget USD 6.00->3.16; gcp-vertex U 76.22->67.99; local now best (U=69.74) |
| P2 email | REDACT -> gcp-vertex | 76.22 | REDACT -> local | 69.74 | budget USD 6.00->3.16; gcp-vertex U 76.22->67.99; local now best (U=69.74) |
| P3 Aadhaar | REDACT -> aws-bedrock | 75.12 | REDACT -> local | 69.74 | budget USD 6.00->3.16; aws-bedrock utility 75.12->64.64; local now best (U=69.74) |
| P4 cloud ARN | ALLOW -> gcp-vertex | 76.22 | REDACT -> local | 69.74 | policy ALLOW->REDACT ; budget USD 6.00->3.16; gcp-vertex U 76.22->67.99; local now best (U=69.74) |
| P5 40 KB log | ALLOW -> gcp-vertex | 76.22 | BLOCK -> NOT_ROUTED | - | policy ALLOW->BLOCK ; budget USD 6.00->3.16 |

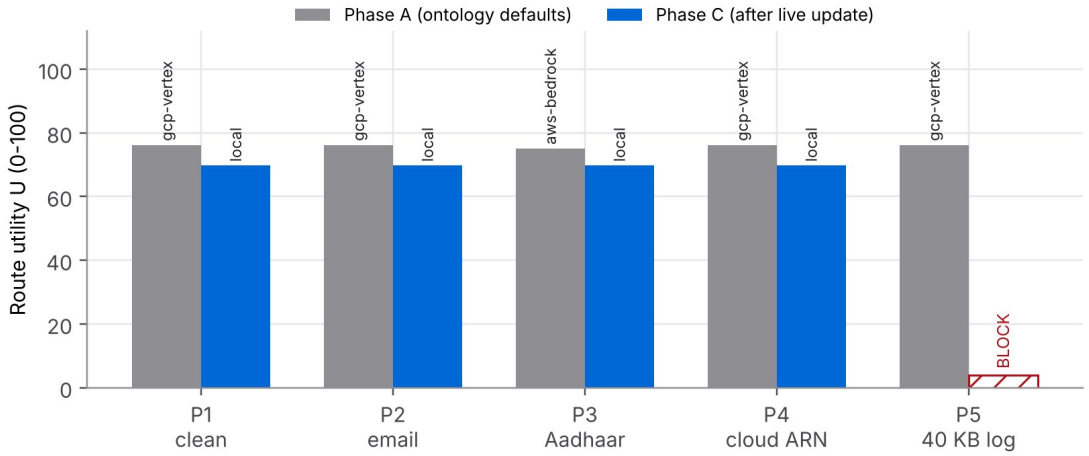



**Figure 5.3:** Route utility and chosen provider per prompt, Phase A versus Phase C

Two live inputs explain all of it, as the leave-one-feed-out re-runs in Table 5.5 confirm:

- **Exchange rate (structured).** INR 300 per 1,000 requests at 94.91 INR/USD is USD 3.1609. Azure (USD 3.30) now fails `OVER_BUDGET`, and the cost score of every cloud provider falls, so the local runtime (USD 0.40) becomes the best feasible choice (U = 69.74).
- **AI incident reports (unstructured text).** The keyword parser found 11 privacy-related reports in 18.9 days of free-text news items, 0.5819 per day, above the 0.5 threshold. Posture ELEVATED adds two redaction patterns, so the AWS ARN in P4 is redacted (R2) instead of sent in clear, and halves the R5 limit to 32,768 bytes, so the 40 KB log dump in P5 is blocked.
- **Not causes today:** the three status feeds reported no active incident for the providers, and the KEV catalog stayed below its calibrated thresholds.

**Table 5.5:** Attribution: each feed removed in turn (same parsed data, re-planned)

| Feed removed | Posture without it | Plan changes that disappear without it |
|---|---|---|
| gcp_status | ELEVATED | none |
| azure_status | ELEVATED | none |
| aws_health | ELEVATED | none |
| cisa_kev | ELEVATED | none |
| fx_usd_inr | ELEVATED | P1 ALLOW->ALLOW gcp-vertex->local; P2 REDACT->REDACT gcp-vertex->local; P3 REDACT->REDACT aws-bedrock->local; P4 REDACT->REDACT gcp-vertex->local |
| aiid_rss | LOW | P4 ALLOW->REDACT; P5 ALLOW->BLOCK local->- |

## 5.6 Phase D: deterministic replay (not live)

Today's provider feeds were quiet, so the provider-status path is shown with a **replay**. The replay reads only the files pinned in `data/replay/`; each is checked against the SHA-256 in `MANIFEST.json` before use, so the result is the same on every run. Four inputs are text-free derivatives of real payloads saved from live runs, and one is a hand-written Azure what-if fixture in the feed's format (marked as such inside the file) that reports an Azure AI Foundry outage in Central India. The demo evaluates them as if the clock read **2026-02-27 13:00 UTC**. At that moment the real GCP feed contains an active Vertex Gemini API incident (status impact SERVICE_INFORMATION, with degradation keywords in its description). The AWS feed and the AI Incident Database contribute **no events** at that date: the AWS events are later and in other regions, and the 100 newest AI incident items all date from October 2026. Table 5.6 shows the plans at each step.

**Table 5.6:** Replay: plans at each step

| Prompt | Baseline | D.1 incidents (Feb 27) | D.2 KEV (Feb 27) | D.3 KEV (Sep 24) |
|---|---|---|---|---|
| P1 clean | ALLOW -> gcp-vertex (76.22) | ALLOW -> aws-bedrock (75.12) | ALLOW -> aws-bedrock (75.12) | ALLOW -> aws-bedrock (75.12) |
| P2 email | REDACT -> gcp-vertex (76.22) | REDACT -> aws-bedrock (75.12) | REDACT -> aws-bedrock (75.12) | REDACT -> aws-bedrock (75.12) |
| P3 Aadhaar | REDACT -> aws-bedrock (75.12) | REDACT -> aws-bedrock (75.12) | REDACT -> aws-bedrock (75.12) | REDACT -> aws-bedrock (75.12) |
| P4 cloud ARN | ALLOW -> gcp-vertex (76.22) | ALLOW -> aws-bedrock (75.12) | ALLOW -> aws-bedrock (75.12) | REDACT -> aws-bedrock (75.12) |

| Prompt | Baseline | D.1 incidents (Feb 27) | D.2 KEV (Feb 27) | D.3 KEV (Sep 24) |
|---|---|---|---|---|
| P5 40 KB log | ALLOW -> gcp-vertex (76.22) | ALLOW -> aws-bedrock (75.12) | ALLOW -> aws-bedrock (75.12) | BLOCK -> NOT_ROUTED |

In step D.1 the GCP parser marks gcp-vertex **degraded** (confidence 0.9). The status individual in the ontology lowers its availability by 0.8 points and multiplies its p95 latency by 1.5, so its utility falls from 76.22 to 71.16 and aws-bedrock wins four prompts. The Azure outage sets `excludesProvider`, so azure-ai is reported as `DISABLED` in every route. In step D.2 the real KEV catalog at the same date shows 29 additions (ratio 1.611) and 9 watched-vendor hits, which is below the calibrated thresholds, so the posture stays LOW and no plan changes. Step D.3 replays the KEV catalog at 2026-09-24, the single HIGH date of the backtest: kev_stack: relevant_hits=12 >= 12.0; kev_volume: ratio=2.043 >= 1.89. Two independent signals are met, so HIGH restricts all traffic to IN and LOCAL, cuts the byte limit to 16,384, redacts the ARN, blocks the log dump and raises a lockdown recommendation for the operator (Listing 7). Figure 5.4 is the provider sub-graph exported by `graph.to_svg()` after step D.2.

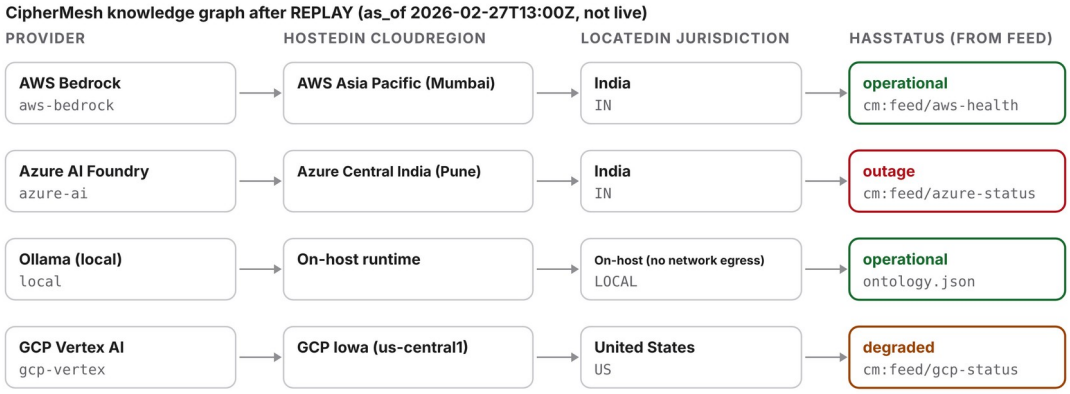



**Figure 5.4:** Provider placement and status sub-graph exported by the agent after the replay

```
21:42:19.677 Replay plans after D.3  (plan version 12, posture HIGH)
21:42:19.678  prompt  decision route  utility  budget USD  residency  R5 bytes
21:42:19.678  P1 clean  ALLOW  aws-bedrock  75.12  6.0000  IN,LOCAL  16384
21:42:19.678  P2 email  REDACT  aws-bedrock  75.12  6.0000  IN,LOCAL  16384
21:42:19.679  P3 Aadhaar  REDACT  aws-bedrock  75.12  6.0000  IN,LOCAL  16384
21:42:19.679  P4 cloud ARN  REDACT  aws-bedrock  75.12  6.0000  IN,LOCAL  16384
21:42:19.679  P5 40 KB log  BLOCK  NOT_ROUTED  -  6.0000  IN,LOCAL  16384
21:42:19.679  context note: threat posture restricts all traffic to ['IN', 'LOCAL']
21:42:19.679  context note: lockdown RECOMMENDED to operator (not applied
      automatically)
```

*Listing 7: replay step D.3 from logs/live_feed_run.log*

Taken together, the replay checks the three adaptation paths that the live run could not show on the day: a degraded provider is penalised but kept, a provider in outage is excluded, and the strictest posture only appears when two independent threat signals agree. Because every replay input is pinned and hash-checked, these results are identical on every run and on any machine, and the run log shows the hash of each file before it is used.

# 6 TESTING

The full suite has **242 tests and all pass**: 112 engine tests from Units 1-3 (unchanged) and 130 new tests for this unit. Table 6.1 summarises the new tests; `logs/pytest_results.txt` lists every test by name.

**Table 6.1:** New tests in tests/knowledge/

| File | Tests | What is checked |
|---|---|---|
| `test_ontology.py` | 15 | Schema passes; six kinds of invalid document rejected; undefined IRI, wrong range, functional violation and sub-class cycle caught; 25 categories, rule codes and provider profiles match the engine |
| `test_graph.py` | 11 | Add / remove with provenance, indexes, conjunctive query, RDFS inference and idempotence, residency equivalence for all 25 categories and 300 pairs, JSON round trip, JSON-LD and SVG export |
| `test_parsers.py` | 52 | Region / service normalisation; each parser on saved samples (a synthetic AI-incident feed with invented wording); time travel; malformed JSON, bad records, corrupt FX values; billion-laughs, XXE and HTML doctype refused; derivatives text-free and signal-identical; no feed prose stored |
| `test_http_fallback.py` | 14 | Local HTTP server: User-Agent and hash, retries, size cap, cache, non-HTTPS refused, guarded redirects, charset, overall deadline, fallback on network failure and on an HTML 200 page, stale snapshot down-weighted, withheld raw payload, tampered snapshot rejected |
| `test_adapter_properties.py` | 21 | PropertySink protocol (also for a belief-base-like class), history, confidence gate, subscribers, concurrent writers, triples with provenance, expiry of old live facts, budget rule, posture thresholds, two-signal HIGH, hysteresis, stale data rejected |
| `test_adaptive_agent.py` | 12 | Graph profiles reproduce engine routing; degraded penalises, outage excludes, recovery restores; FX budget; ELEVATED and HIGH policies; REVIEW local only; deterministic replay; pinned inputs match manifest; timing; engine sync; unsafe custom pattern rejected |
| `test_calibration.py` | 4 | Ontology thresholds equal the backtest calibration; posture shares bounded; tie handling; two-signal HIGH, hysteresis and attribution |
| `test_live_endpoints.py` | 1 | Marked `live`: contacts all six endpoints and checks status, hash and properties; skipped automatically without network |

Edge cases were chosen from the failure modes of real feeds: an empty RSS channel (Azure today), an HTML error page served with status 200, a redirect to another host, a UTF-16 body with a byte-order mark (AWS), incidents with no location (prose fallback), resolved events without an end time, KEV records with bad dates, exchange rates outside a sanity band, and a snapshot whose bytes no longer match its recorded hash. The live test ran and passed during the recorded test run because the network was available.

# 7 SUBMISSION ANALYSIS

Table 7.1 checks each requirement against its evidence.

**Table 7.1:** Requirement status

| Requirement | Evidence | Status |
|---|---|---|
| Structured knowledge format | `ontology.json` + schema; 1,170 triples; validation tests | **PASS** |
| Live REST API integration | 6/6 endpoints HTTP 200 with hashes and snapshots | **PASS** |
| Parse live unstructured data | Azure RSS prose, GCP descriptions, AWS messages, AI-incident news text (keyword classes) | **PASS** |
| Properties change instantly | 27 live property changes; property to plan under 1 ms | **PASS** |
| Plans adapt from live unstructured data | Attribution: without the AI-incident text feed, P4 and P5 revert (posture LOW); budget changes come from the structured FX feed | **PASS** |
| Threat signal calibrated | Backtest of 122 dates; HIGH 0.8 % of days (was 46 %) | **PASS** |
| Residency derivable from graph | Equivalence tests and log line `EQUAL=True` | **PASS** |
| Tests | 242 passed (130 new) | **PASS** |

## 7.1 Strengths

- One source of truth: providers, regions, legal links, rule order, status penalties, posture thresholds and the budget all live in a validated file, and the router's residency rule is derived from it, not duplicated.
- Provenance everywhere: every triple and every property records its source, time and confidence; replayed data is labelled as replay in the graph, the properties and the log.
- Defensive integration: TLS verification, size caps, retries, DTD refusal, BOM-aware decoding, hash-checked snapshots and an offline mode.
- Explainable adaptation: each plan change in the log has a reason computed from the before and after plans.

## 7.2 Limitations

- Provider prices, latencies and quality scores remain illustrative planning numbers, as in Units 3.1 and 3.2. The INR 300 budget is an illustrative organisational setting.
- The AWS endpoint backs the public Health Dashboard page and is not a documented API; if its format changes the connector falls back to the saved snapshot.
- The live posture change rests on one weak signal: keyword classes over news text. It can miss unusual wording or other languages (the feed carries Arabic, Spanish and Chinese reports) and it cannot be backtested, so its threshold is a judgement value. It can reach ELEVATED alone but never HIGH, and lockdown stays operator-only.
- The ontology uses a JSON-LD style and RDFS-level inference implemented in Python; it is not processed by a full OWL reasoner.
- Provider-status adaptation was not visible live on the day of the run, so it is demonstrated with the labelled replay; the Azure item in that replay is a written fixture.

### 7.3 How this feeds the Capstone (5.2)

The Capstone combines the BDI layer of 4.1 with this knowledge layer. Because `KnowledgeAdapter` writes through the `PropertySink` interface, the BDI belief base can be passed in directly: a live Vertex AI incident becomes a belief, the BDI agent reconsiders an intention that depends on gcp-vertex, and the knowledge graph keeps the evidence. `sync_engine_agent` already pushes posture-driven policy into a running `CipherMeshAgent`, so the network server of Unit 2.2 can serve the adapted policy without restarting.

## 8 CONCLUSION

CipherMesh now knows *where* its providers are, *which law* applies to which data, and *how* the outside world looks right now. The ontology stores this knowledge formally and can be validated and reasoned over; the six connectors bring in real provider-health, threat and market data from public REST and RSS endpoints [15] [16]; and the adapter turns that data into properties that change the agent's plan in well under a millisecond. In the recorded live run, the exchange rate moved the budget and keyword parsing of the AI Incident Database's free-text reports [17] raised the threat posture, so all five test prompts were planned differently; leave-one-feed-out re-runs confirm both causes. The CISA KEV catalog [18] stayed below thresholds calibrated on a one-year backtest. The pinned replay showed the provider-health path using real status data from the Google Cloud [19], AWS [20] and Azure [21] feeds. Every change is explained, logged with provenance and covered by tests.

# REFERENCES

[1] M. Sporny, D. Longley, G. Kellogg, M. Lanthaler, P.-A. Champin, and N. Lindström, "JSON-LD 1.1: A JSON-based serialization for linked data," W3C Recommendation, Jul. 16, 2020. [Online]. Available: https://www.w3.org/TR/json-ld11/ (accessed Oct. 31, 2026).

[2] W3C OWL Working Group, "OWL 2 Web Ontology Language document overview (second edition)," W3C Recommendation, Dec. 11, 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/ (accessed Oct. 31, 2026).

[3] D. Brickley and R. V. Guha, "RDF Schema 1.1," W3C Recommendation, Feb. 25, 2014. [Online]. Available: https://www.w3.org/TR/rdf-schema/ (accessed Oct. 31, 2026).

[4] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199-220, 1993.

[5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[6] A. Wright, H. Andrews, B. Hutton, and G. Dennis, "JSON Schema: A media type for describing JSON documents," IETF Internet-Draft draft-bhutton-json-schema-01 (JSON Schema 2020-12), Jun. 2022.

[7] Government of India, *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023), Gazette of India, Ministry of Law and Justice, Aug. 11, 2023.

[8] Government of India, *The Aadhaar (Targeted Delivery of Financial and Other Subsidies, Benefits and Services) Act, 2016* (Act No. 18 of 2016).

[9] European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, L 119, pp. 1-88, May 4, 2016.

[10] T. Bray, Ed., "The JavaScript Object Notation (JSON) data interchange format," IETF RFC 8259, Dec. 2017.

[11] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," IETF RFC 9110, Jun. 2022.

[12] OWASP Foundation, "XML External Entity prevention cheat sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html (accessed Oct. 31, 2026).

[13] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in *Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS-95)*, San Francisco, CA, USA, 1995, pp. 312-319.

[14] European Central Bank, "Euro foreign exchange reference rates." [Online]. Available: https://www.ecb.europa.eu/stats/policy_and_exchange_rates/euro_reference_exchange_rates/html/index.en.html; served through the Frankfurter API, https://frankfurter.dev/ (accessed Oct. 31, 2026).

[15] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[16] RSS Advisory Board, "RSS 2.0 specification," version 2.0.11, Mar. 30, 2009. [Online]. Available: https://www.rssboard.org/rss-specification (accessed Oct. 31, 2026).

[17] S. McGregor, "Preventing repeated real world AI failures by cataloging incidents: The AI Incident Database," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 17, 2021, pp. 15458-15463.

[18] Cybersecurity and Infrastructure Security Agency, "Known Exploited Vulnerabilities catalog." [Online]. Available: https://www.cisa.gov/known-exploited-vulnerabilities-catalog (accessed Oct. 31, 2026).

[19] Google Cloud, "Google Cloud Service Health," incidents.json feed. [Online]. Available: https://status.cloud.google.com/ (accessed Oct. 31, 2026).

[20] Amazon Web Services, "AWS Health Dashboard." [Online]. Available: https://health.aws.amazon.com/health/status (accessed Oct. 31, 2026).

[21] Microsoft, "Azure status," RSS feed. [Online]. Available: https://azure.status.microsoft/en-us/status/ (accessed Oct. 31, 2026).

# APPENDIX A HOW TO RUN

These are the same commands as in the README. The report itself is built with the shared course report kit (`dsl.js`, `build_report.js`, `render_diagrams.js`), which is not part of this folder; `docs/report_src/README.md` explains how to rebuild it.

```
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt            # includes pytest

# live demonstration: phases A-D, writes logs/ and data/snapshots/
python demo/live_adaptation_demo.py

# same run without network (saved snapshots only, labelled OFFLINE)
python demo/live_adaptation_demo.py --offline --logs /tmp/a42_offline_logs

# KEV backtest and threshold check -> logs/kev_backtest.json
python demo/kev_backtest.py

# re-pin the replay inputs (only when deliberately updating them)
python demo/pin_replay_inputs.py

# full test suite; the live test is skipped automatically when offline
python -m pytest -o addopts= -v -rs tests
python -m pytest -m live -v          # only the live endpoint test

# regenerate figures (needs matplotlib and the report kit)
python3 docs/figures/make_charts.py <reportkit>
python3 docs/figures/make_diagrams.py
```

*Listing 8: commands*

# APPENDIX B FILE LISTING

Table B.1 lists the files in the submission folder.

**Table B.1:** Deliverable files

| Path | Contents |
|---|---|
| `ciphermesh/knowledge/` | ontology.json, ontology.schema.json, ontology.py, graph.py, properties.py, adapter.py, posture.py, calibration.py, adaptive_agent.py, data/whatif_azure_status.xml |
| `ciphermesh/knowledge/feeds/` | base.py, safe_xml.py, normalize.py, status_common.py, gcp_status.py, azure_status.py, aws_health.py, cisa_kev.py, fx_rates.py, ai_incidents.py |
| `ciphermesh/` (other modules) | Engine from Units 1-3 (unchanged) |
| `demo/` | live_adaptation_demo.py (phases A-D), kev_backtest.py, pin_replay_inputs.py |
| `tests/knowledge/` + `fixtures/` | 130 tests; saved samples of each feed (synthetic wording for AI incidents) |
| `tests/engine/` | 112 engine tests |
| `data/snapshots/` | Latest live data: text-free derivatives (raw only for FX) + .meta.json with the original URL, status, bytes, SHA-256, fetched_at |
| `data/replay/` | Pinned replay inputs + MANIFEST.json (SHA-256 of each file) |
| `logs/live_feed_run.log` | Timestamped run log (phases A-D) |
| `logs/adaptation_events.jsonl` | One JSON line per adaptation with property changes and timings |
| `logs/live_endpoints_report.json` | Per-endpoint status, latency, bytes, hash, facts |
| `logs/demo_record.json`, `knowledge_graph_export.json`, `knowledge_graph_*.svg` | Plans, properties, attribution, graph export with provenance, graph figures |
| `logs/kev_backtest.json` | One-year KEV backtest and calibrated thresholds |
| `logs/pytest_results.txt` | Full test output |
| `docs/Assignment_4.2_Knowledge_Representation_Adaptive_API_Report.docx/.pdf` | This report |
| `docs/figures/`, `docs/report_src/content.js` | Figures with their sources; report source |